

Clasificación de Imágenes de Gatos y Perros mediante Redes Neuronales Convolucionales

Alejandra Nazareth Sosa Carrillo José Fernando Chalo Sajquín
 Manejo de Datos para la Inteligencia Artificial
 Universidad Galileo, Ciudad de Guatemala, Guatemala
 nazareth.sosa@galileo.edu jose.chalo@galileo.edu

Resumen—Este trabajo presenta el desarrollo de un clasificador binario de imágenes capaz de distinguir entre gatos y perros empleando una Red Neuronal Convolutiva (CNN) entrenada desde cero. El dataset utilizado es el Microsoft Cats vs Dogs, obtenido de Kaggle, el cual fue sometido a un proceso exhaustivo de limpieza que incluyó la eliminación de archivos corruptos, duplicados e imágenes de baja resolución. El preprocesamiento incorporó normalización de píxeles y técnicas de *Data Augmentation* para mejorar la capacidad de generalización del modelo. La arquitectura propuesta consta de tres bloques convolucionales con *Batch Normalization*, *Max Pooling* y *Dropout*, seguidos de una capa densa de clasificación con activación Sigmoid. El entrenamiento fue controlado mediante *Early Stopping* y reducción adaptativa de la tasa de aprendizaje. Los resultados obtenidos demuestran que la arquitectura implementada alcanza un desempeño competitivo en la tarea de clasificación visual binaria sin recurrir a *transfer learning* ni modelos pre-entrenados.

I. INTRODUCCIÓN

LA clasificación de imágenes es una de las tareas más importantes dentro de la visión por computadora, ya que permite asignar automáticamente una categoría a una imagen. Este tipo de sistemas se utiliza en múltiples áreas como salud, conducción autónoma, vigilancia y análisis de contenido digital.

Las Redes Neuronales Convolucionales (CNN) son uno de los enfoques más utilizados para este problema. Estas redes están diseñadas para aprender patrones espaciales en imágenes, detectando características desde bordes simples hasta estructuras complejas. Gracias a esta capacidad, se han convertido en una de las técnicas estándar para tareas de visión por computadora [1].

El problema *Cats vs Dogs* es un conjunto de datos muy utilizado en aprendizaje automático. Fue popularizado a través de una competencia en Kaggle, donde el objetivo es clasificar imágenes entre gatos y perros [2]. Aunque el problema parece simple por ser binario, en la práctica es difícil debido a variaciones en iluminación, postura, fondo y calidad de las imágenes.

Este tipo de datasets se utiliza como benchmark para evaluar modelos de clasificación, ya que permite medir la capacidad de generalización de una red neuronal en condiciones reales [3]. En este trabajo se desarrolla una solución basada en una red neuronal convolutiva entrenada desde cero, aplicando técnicas de preprocesamiento y aumento de datos para mejorar el rendimiento del modelo.

El resto del documento se organiza de la siguiente forma: la Sección II describe la metodología utilizada, la Sección III presenta los resultados obtenidos, y la Sección IV expone las conclusiones del estudio.

II. METODOLOGÍA

II-A. Dataset

El dataset utilizado es el Microsoft Cats vs Dogs, accedido mediante la API de kagglehub [5]. La colección se organiza en dos subdirectorios, *Cat* y *Dog*, cada uno con imágenes JPEG de resolución y contenido variable. La Tabla I resume la composición inicial antes de la limpieza.

Cuadro I
COMPOSICIÓN DEL DATASET ANTES DE LA LIMPIEZA.

Clase	Cantidad de imágenes
Cat	12501
Dog	12501
Total	25002

II-B. Análisis Exploratorio de Datos

Antes de aplicar cualquier paso de limpieza, el dataset fue examinado para caracterizar su composición e identificar posibles problemas de calidad. Se contabilizó el número de imágenes por clase para evaluar el balance entre categorías y se visualizó una muestra aleatoria de ambas clases para inspeccionar calidad, modo de color y variabilidad.

Adicionalmente, el ancho y el alto de todas las imágenes fueron analizados mediante histogramas y un gráfico de dispersión de resoluciones. Este análisis reveló una amplia variabilidad dimensional, con presencia de imágenes muy pequeñas y muy grandes, lo que motivó la etapa de eliminación por resolución mínima.

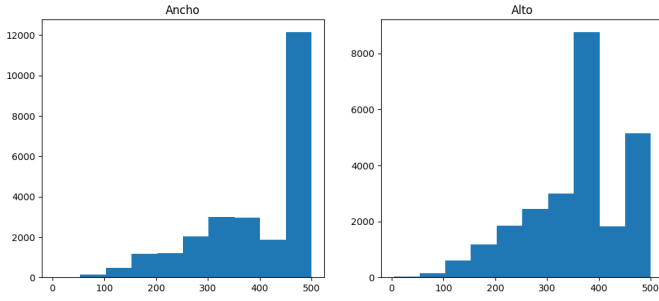


Figura 1. Histogramas del ancho (izquierda) y el alto (derecha) de las imágenes del dataset.

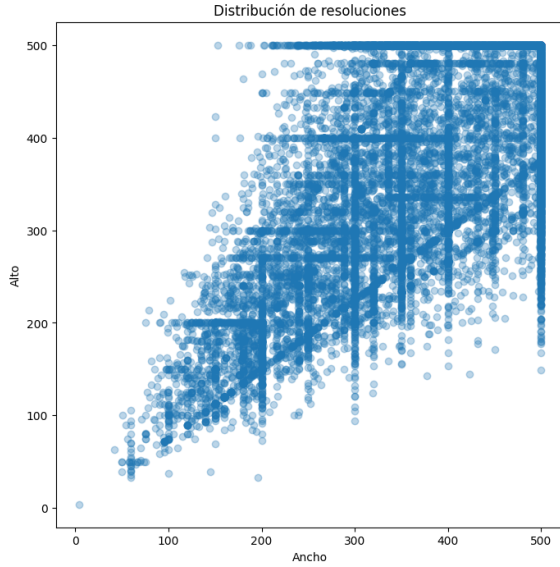


Figura 2. Dispersión de resoluciones: ancho vs. alto de cada imagen.

II-C. Limpieza de Datos

Se implementó un pipeline de limpieza en tres etapas para eliminar imágenes que introducirían ruido o errores durante el entrenamiento.

Imágenes corruptas. Cada imagen fue abierta con PIL y verificada con `img.verify()`, seguida de un intento de decodificación con `tf.image.decode_image`. Los archivos que fallaran en cualquiera de las dos etapas fueron identificados como corruptos y eliminados.

Duplicados perceptuales. Los duplicados se detectaron con el algoritmo *Perceptual Hash* (pHash) de la librería `imagehash` [?]. Cuando dos imágenes compartían el mismo hash, la segunda ocurrencia era marcada y eliminada. Adicionalmente, dos imágenes (`Cat/11565.jpg` y `Cat/8456.jpg`) fueron eliminadas manualmente por no corresponder a gatos reales.

Imágenes de baja resolución. Las imágenes con alguna dimensión inferior a 100 píxeles fueron eliminadas por contener información visual insuficiente para el modelo. La Tabla II resume los resultados de cada etapa.

Cuadro II
RESUMEN DEL PIPELINE DE LIMPIEZA DE DATOS.

Etapa	Eliminadas
Imágenes corruptas	11
Duplicados (incl. 2 manual)	38
Baja resolución (< 100 px)	179
Imágenes válidas restantes	24774

II-D. Preprocesamiento y Data Augmentation

Después de la limpieza del dataset, las imágenes se cargaron usando `image_dataset_from_directory` de TensorFlow. El conjunto se dividió automáticamente en 80 % para entrenamiento y 20 % para validación. Todas las imágenes se redimensionaron a 128×128 píxeles y se trabajó con batches de 32.

Para mejorar la generalización del modelo, se aplicó *data augmentation* solo en el conjunto de entrenamiento. La idea es generar variaciones de las imágenes durante el entrenamiento para evitar overfitting.

Las transformaciones incluyen normalización de píxeles a rango $[0, 1]$, volteo horizontal aleatorio, cambios leves de brillo y variaciones de contraste. El conjunto de validación solo se normaliza, sin augmentations.

Luego, el dataset se optimizó con *shuffle* y *prefetch* para hacer el entrenamiento más eficiente.

II-E. Arquitectura de la CNN

El modelo es una CNN secuencial con tres bloques convolucionales. Cada bloque tiene una capa Conv2D, Batch Normalization, Max Pooling y Dropout.

Los bloques usan 32, 64 y 128 filtros respectivamente. Después de esto, la salida se aplanar y pasa a una capa Dense de 256 neuronas, seguida de Batch Normalization y Dropout. Finalmente, una neurona con activación Sigmoid realiza la clasificación binaria.

La estructura completa se resume en la Tabla III.

Cuadro III
ARQUITECTURA DE LA CNN. BN = BATCH NORMALIZATION.

Bloque	Capas	Filtros	Drop
1	Conv2D→BN→MaxPool	32	0.25
2	Conv2D→BN→MaxPool	64	0.25
3	Conv2D→BN→MaxPool	128	0.25
Cabeza	Flatten→Dense→BN→Sigmoid	256/1	0.50

II-F. Configuración de Entrenamiento

El modelo se entrenó usando el optimizador Adam y la función de pérdida Binary Crossentropy. Se ejecutó por un máximo de 30 epochs.

Se usaron dos callbacks principales: *EarlyStopping* para detener el entrenamiento cuando la validación deja de mejorar, y *ReduceLROnPlateau* para reducir la tasa de aprendizaje cuando el rendimiento se estanca.

Los hiperparámetros principales se resumen en la Tabla IV.

Cuadro IV
CONFIGURACIÓN DEL ENTRENAMIENTO.

Parámetro	Valor
Tamaño de imagen	128 × 128 px
Batch size	32
Optimizador	Adam
Función de pérdida	Binary Crossentropy
Épocas máximas	30
División de validación	20 %
Semilla aleatoria	123
Dropout (bloques conv.)	0.25
Dropout (cabeza)	0.50
Paciencia EarlyStopping	4 épocas
Factor ReduceLROnPlateau	0.5

III. RESULTADOS

III-A. Curvas de Entrenamiento

La Figura 3 muestra la evolución de la exactitud y la pérdida sobre los conjuntos de entrenamiento y validación por época. El uso de *EarlyStopping* garantizó que se preservara el mejor checkpoint independientemente del momento en que el entrenamiento finalizara.

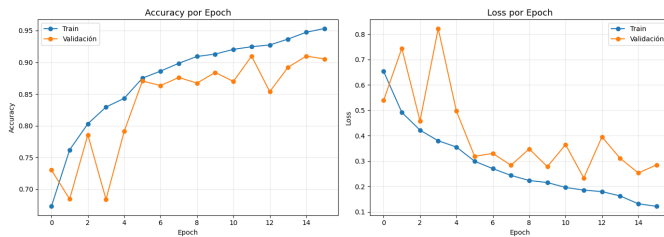


Figura 3. Accuracy (izquierda) y Loss (derecha) de entrenamiento y validación por época.

III-B. Matriz de Confusión

La Figura 4 y la Tabla V resumen las clasificaciones del modelo sobre el conjunto de validación con umbral de decisión 0.5.

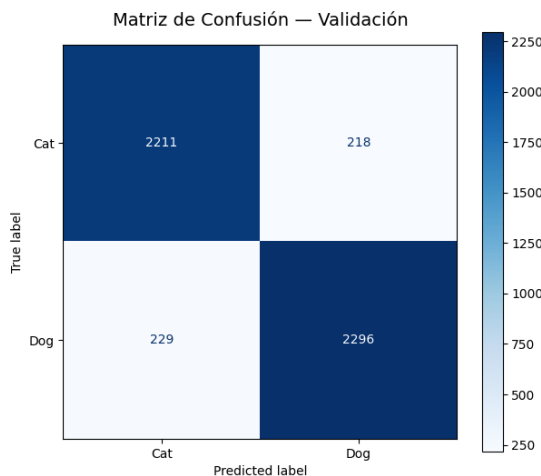


Figura 4. Matriz de confusión sobre el conjunto de validación.

Cuadro V
VALORES DE LA MATRIZ DE CONFUSIÓN.

Categoría	Cantidad
Verdaderos Negativos (Cat → Cat)	2211
Falsos Positivos (Cat → Dog)	218
Falsos Negativos (Dog → Cat)	229
Verdaderos Positivos (Dog → Dog)	2296

III-C. Curva ROC

La Figura 5 evalúa la capacidad discriminativa del modelo a través de todos los umbrales posibles. El Área Bajo la Curva (AUC-ROC) cuantifica esta capacidad: 1.0 representa un clasificador perfecto y 0.5 uno aleatorio.

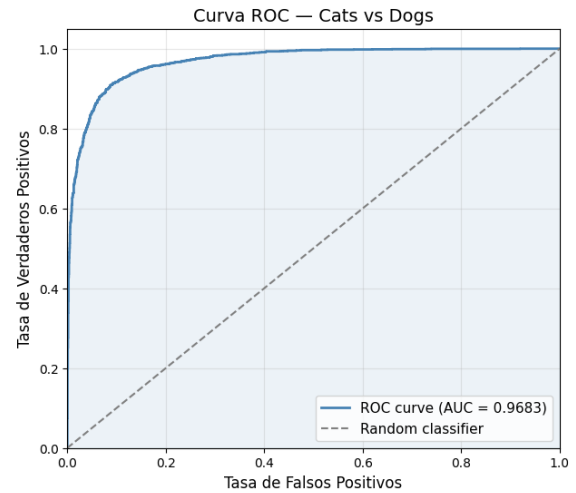


Figura 5. Curva ROC y puntuación AUC sobre el conjunto de validación.

III-D. Métricas Finales

La Tabla VI presenta el conjunto completo de métricas calculadas sobre validación. Precisión, Recall y F1-Score se calcularon con umbral 0.5, el AUC-ROC proviene de las probabilidades continuas de la capa Sigmoid.

Cuadro VI
MÉTRICAS DE EVALUACIÓN FINALES.

Métrica	Valor
Accuracy	0.9098
Precision	0.9133
Recall	0.9093
F1-Score	0.9113
AUC-ROC	0.9683
Validation Loss	0.2339
Mejor época (EarlyStopping)	12

III-E. Predicciones de Muestra

La Figura 6 muestra imágenes del conjunto de validación junto con sus etiquetas reales y las predicciones del modelo. Los títulos en verde indican predicciones correctas y en rojo, errores.



Figura 6. Predicciones sobre imágenes de validación. Verde = correcto, rojo = error.

IV. CONCLUSIONES

Este trabajo presentó un pipeline completo para la clasificación binaria de imágenes utilizando una CNN entrenada desde cero sobre el dataset Microsoft Cats vs Dogs. El proyecto abarcó adquisición de datos, evaluación de calidad, limpieza en tres etapas, preprocesamiento con augmentación, diseño de arquitectura y evaluación rigurosa.

La fase exploratoria reveló que el dataset crudo contenía archivos corruptos, duplicados perceptuales e imágenes de resolución insuficiente, todos eliminados sistemáticamente antes de cualquier proceso de aprendizaje. Esta atención a la calidad de los datos es un prerequisite para una evaluación confiable del modelo.

La estrategia de preprocesamiento normalización de píxeles combinada con volteo aleatorio, variaciones de brillo y contraste expuso la red a mayor variedad de entradas sin alterar las etiquetas de verdad, contribuyendo a reducir la brecha entre entrenamiento y validación junto con el *Batch Normalization*.

Los resultados confirman que una arquitectura CNN moderadamente profunda, combinada con regularización apropiada y control mediante callbacks, alcanza buena generalización en este problema sin necesidad de *transfer learning*. Como trabajo futuro, explorar modelos pre-entrenados más grandes, *Test-Time Augmentation* o mecanismos de atención podría ofrecer ganancias adicionales en el desempeño.

REPOSITORIO DEL PROYECTO

El código fuente completo y el paper científico están disponibles públicamente. El código QR permite acceder directamente al paper.



Figura 7. Escanea para consultar el paper científico.

REFERENCIAS

- [1] NVIDIA, "What is a Convolutional Neural Network?," NVIDIA Developer Blog. Disponible en: <https://www.nvidia.com/en-us/glossary/convolutional-neural-network/>
- [2] Kaggle, "Dogs vs. Cats Competition," 2013. Disponible en: <https://www.kaggle.com/c/dogs-vs-cats>
- [3] Microsoft Research, "Asirra: A CAPTCHA that Exploits Interest-Aligned Image Recognition," Disponible en: <https://www.microsoft.com/en-us/research/wp-content/uploads/2007/10/CCS2007.pdf>
- [4] I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*. MIT Press, 2016. [En línea]. Disponible: <https://www.deeplearningbook.org>
- [5] W. Cukierski, *Cats vs. Dogs*, Kaggle, 2018. [En línea]. Disponible: <https://www.kaggle.com/datasets/shaunthesheep/microsoft-catsvsdogs-dataset/data>
- [6] J. Buchner, *ImageHash: A Python Perceptual Image Hashing Module*, 2013. [En línea]. Disponible: <https://github.com/JohannesBuchner/imagehash>